taking a name class from a first name, last name, and so on. The value objects are super-fast to test and tend to attract logic.

- In tests, I try to create as few objects as possible, touch the database as little as possible, and be aware of dependencies and places to mock that can limit the amount of setup a test needs.
- RSpec has a `—profile` option that shows you the slowest tests in a test run. You need to be careful with it (often the slow test will be the one that had the garbage collector run during it), but it's a good thing to run now and then to try to speed up your slowest tests. The same configuration option that allows the use of `--only-failures` keeps a running list of the runtimes of every spec in the system. This can be useful to look at.
- All that said, don't despair if you can't make this all work immediately. I've never quite been able to make an entire project work using the ActiveRecord-specific test helper. My main problem is not making an individual test run, and it's not keeping track of dependencies manually. Rather, my main problem has typically been integrating the separate sets of tests into the entire ecosystem of the project—that includes my ability to run the entire spec suite, other developers' ability to run the suite, extra continuous-integration setup, and the like. Being aware of dependencies (and using Spring) provides most of the benefit with a fraction of the fuss. Still, it's probably something I could be more aggressive about; most of my codebases wind up with test suites that are slower than I'd like.

What You've Done

In this chapter you explored different solutions for increasing the speed of your tests. Test speed can be important in a lot of ways. It's valuable to be able to run useful tests in development without losing your focus. It's also nice to be able to run all your tests before deploying code and not have that run take a half hour when there's a bug in production you're scrambling to fix.

You learned about strategies on the command line to run only a subset of tests at once to allow you to run them more quickly and to focus on the code that is currently being changed. You also learned about strategies to make the test suite faster in a larger sense by allowing you to isolate code from Rails or by removing dependencies so that tests don't touch the database.

A lot of this advice is particularly difficult to apply to legacy codebases that have not been built to support these ideas. The next chapter talks about strategies for dealing with the particular issues of legacy code.